

Evaluation des performances des systemes informatiques

Cours 09 – Simulateur

Pedro Braconnot Velloso & Abdelhak Hidouri

Au programme de ce cours

- Generation de nombres aleatoires
 - En C++, Python ou Java
- Construire notre propre simulateur
 - Simulateur oriente objet
- Simuler un systeme simple

A la fin de ce cours, vous serez capable de :

1. Comprendre l'architecture d'un simulateur a evenements discrets
2. Implementer chaque composant (classe)
3. Simuler des files d'attente $M/M/1$, $M/M/1/K$, $M/M/c/K$

Le simulateur a evenements discrets

Simuler un modele probabiliste implique

- **Generer** les mecanismes stochastiques du modele
 - Les variables aleatoires du TP precedent
- **Observer** le flot resultant du modele au cours du temps
 - Suivre l'evolution de l'etat du systeme
- **Calculer** les metriques d'interet
 - Temps d'attente, nombre de requetes, debit. . .
- **Exemple** : un client envoie une requete a un serveur web

Le temps n'avance pas de facon continue. Il **saute** d'un evenement au suivant.

Entre deux evenements, rien ne change. On ne simule que les **instants ou quelque chose se passe**.

Exemples d'evenements :

- Arrivee d'un message
- Debut du service
- Fin du service (depart)

→ C'est tres efficace : au lieu de simuler 1 000 000 d'instants, on ne simule que les quelques milliers d'evenements.

Elements cles : variables et evenements

- On suit certaines **variables** :
 - Le temps ecoule depuis le debut de la simulation (t)
 - Les **variables compteurs** (nb arrivees, nb departs...)
 - Les **variables d'etat** (nb clients dans la file, serveur occupe?)
- A chaque **evenement** :
 - Ces variables peuvent changer ou etre mises a jour

Variable	Mise a jour quand ?
t	A chaque evenement
$N_{systeme}$	Arrivee ou depart
N_{file}	Arrivee ou depart
W_{total}	A chaque depart

Pour savoir **quoi faire** quand un evenement se produit.

Exemple : une arrivee se produit.

- Si le serveur est **libre** → commencer le service immediatement, programmer un evenement DEPART
- Si le serveur est **occupe** → mettre en file d'attente
- Si la file est **pleine** (M/M/1/K) → rejeter le message

→ Le simulateur est essentiellement une boucle : extraire l'evenement suivant, mettre a jour les variables, programmer les prochains evenements.

Organisation des evenements

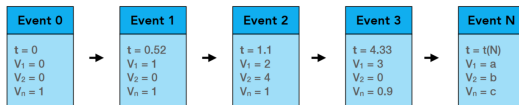
- Les evenements sont stockes dans une **liste ordonnee**
- Organises par le **scheduler** (ordonnanceur)
- Le temps courant t = toujours le temps de l'evenement courant

Son role : maintenir la liste des evenements futurs, **toujours triee par ordre chronologique**.

Methodes principales :

- AddEvent(e) – insere au bon endroit
- GetEvent() – extrait le prochain
- GetCurrentTime() – retourne t

Attention : il faut eviter sort() – trop couteux.



Chaque evenement contient :

- Son **horodatage** (quand il se produit)
- Son **type** (SEND, RECV, DEPT...)
- Les **valeurs des variables** a cet instant

Construire un simulateur

Composants principaux du simulateur

6 composants principaux

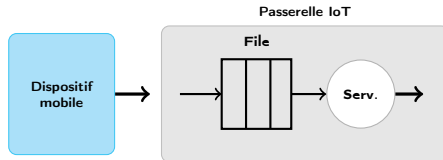
- **Engine (Main)** – moteur de simulation
- **Scheduler** – ordonnanceur d'évenements
- **Client** – generateur de messages
- **Gateway** – passerelle IoT (le systeme)
- **Server** – serveur de traitement
- **Queue** – file d'attente

Chaque composant = une classe.

Un fichier par classe.

Classes auxiliaires :

- **Event** – represente un evenement
- **Message** – represente un message IoT



Chaque composant encapsule ses données et son comportement. Facile d'ajouter des serveurs, changer la file, modifier le client **sans toucher au reste du code**.

C'est exactement comme ns-3 ou OMNeT++ – les grands simulateurs réseaux utilisent cette architecture.

3 types d'evenements

- **SEND_MSG**

- Le client envoie un message a la passerelle
- Quand un message est envoye :
 - Il met **1 seconde** a arriver a la passerelle
 - Un evenement RECV_MSG est programme a $t + 1$

- **RECV_MSG**

- La passerelle recoit le message
- Si serveur libre : service commence, programmer MSG_DEPT
- Si serveur occupe : mettre en file

- **MSG_DEPT**

- La passerelle termine le traitement d'un message
- Si file non vide : servir le suivant

Chaque message transporté contient :

- **messageID** – identifiant unique
- **source** – noeud emetteur
- **destination** – noeud destinataire
- **timestamp** – heure de creation

Inter-arrivees : exponentielle de moyenne λ

$$t_{arrivee} = -\lambda \cdot \ln(U)$$

Temps de service : exponentielle de moyenne μ

$$t_{service} = -\mu \cdot \ln(U)$$

En Python : `rng.exponential(scale=lam)`
`rng.exponential(scale=mu)`

Ce que l'on mesure

- **Nombre instantane**
 - Requetes dans le systeme et dans la file a l'instant t
- **Nombre total de requetes**
 - Arrivees totales, services termines, messages rejetes
- **Nombre moyen de requetes**
 - $\bar{N}_{systeme}$ et $\bar{N}_{file}$ (moyenne temporelle)
- **Temps d'attente**
 - Temps moyen en file $\bar{W}_{file}$
 - Temps moyen dans le systeme $\bar{W}_{systeme}$

Le nombre dans le systeme n'est pas constant. On calcule la **moyenne ponderee par le temps** :

$$\bar{N} = \frac{1}{T} \int_0^T N(t) dt \approx \frac{\sum_i N_i \cdot \Delta t_i}{T}$$

A chaque evenement, on accumule $N \cdot \Delta t$.

$$L = \lambda \cdot W$$

Si vous avez $\bar{N}$ et λ , vous obtenez $\bar{W}$ directement. C'est une verification puissante de votre simulateur.

Methodologie de test

- Methode **Print** par classe
 - Affiche tous les membres de la classe
 - `Print_Message()`, `Print_Event()`, etc.
- Une **fonction de test** par classe dans Engine
 - `Test_Message()`, `Test_Event()`, etc.
 - Cree un objet, le configure, l'affiche

Implementez et testez **une classe a la fois**. Ne passez a la suivante que quand la precedente est validee. C'est le principe du **developpement incremental**.

Role de l'Engine

- Contient la fonction **Main()**
- Stocke les parametres principaux :
 - Nombre de clients
 - Nombre de serveurs dans la passerelle
 - Duree de simulation
- Une seule passerelle dans notre modele
- Methodes :
 - **GenerateTrace(Event e)** – (e.g. : trace CSV, traces, etc.)
 - **CreateClients(n)** – cree les clients
 - **Run()** – boucle principale de simulation
 - **Etc.**

Après chaque evenement : generer la trace

La trace enregistre **chaque evenement** avec son contexte complet. Elle permet de :

- Deboguer le simulateur (verifier l'ordre des evenements)
- Calculer des metriques a posteriori
- Visualiser le comportement du systeme

time	Temps de l'evenement
node	Noeud concerne (0=gateway, N=client)
event	Type : SEND, RECV, DEPT
source	Noeud emetteur
dest	Noeud destinataire
msgID	Identifiant du message

time	node	event	src	dst	msgID
1.202	1	SEND	1	0	1
1.916	0	RECV	1	0	1
2.320	3	SEND	3	0	2
2.391	0	RECV	3	0	2
4.572	0	DEPT	1	0	1
5.916	0	DEPT	3	0	2

→ Lisez : a $t = 1.202$, le client 1 envoie le message 1. A $t = 1.916$, la passerelle le recoit ($\Delta t = 0.714$: temps de propagation). A $t = 4.572$, le service du message 1 se termine.

Membres et methodes

- **Membres**

- EventID – identifiant unique
- Message – le message associe
- EventType – SEND_MSG, RECV_MSG, MSG_DEPT
- EventTime – horodatage de l'evenement

- **Methodes**

- Constructeur : Event(id, type, time, msg)
- SetEventTime(t) / GetEventTime()
- SetEventType(t) / GetEventType()
- PrintEvent()

Membres et methodes

- **Membres**

- `messageID` – identifiant unique
- `source` – noeud emetteur
- `destination` – noeud destinataire
- `timestamp` – heure de creation

- **Methodes**

- Constructeur : `Message(id, src, dst)`
- Getters : `GetMessageID()`, `GetSource()`...
- Setters : `SetTimestamp(t)`...
- `PrintMessage()`

→ Le timestamp du message est l'heure a laquelle il a ete cree. A sa sortie du systeme, $t_{depart} - t_{creation}$ donne le **temps de sejour total** dans le systeme.

Role et methodes

- Les evenements sont stockes en **ordre chronologique**
- Methodes :
 - `AddEvent(e)` – insere au bon endroit
 - `GetEvent()` – extrait le prochain evenement
 - `GetCurrentTime()` – retourne t courant
 - `HasEvents()` – verifie si la liste est vide
- Les evenements doivent etre inseres **directement au bon endroit**
 - On ne peut pas utiliser `sort()` a chaque insertion

- Implementation **en classe** : 100% de la note
- Implementation **a la maison** : 50% de la note

Il est donc fortement recommande d'avancer au maximum pendant les seances.

- Fonctionnement correct du simulateur
- Trace generee et coherente
- Metriques calculees et comparees a M/M/1 theorique
- Code propre et bien commente

Planning sur 3 seances

Seance 1 :

- Main + Trace + Methodes de test
- Classe Message
- Classe Event
- Classe Scheduler

Seance 2 :

- Main + Methodes de test
- Classe Client
- Classe Queue
- Classe Server

Seance 3 :

- Main + Methodes de test
- Classe Engine
- Classe Gateway
- Simulations et resultats

Simulations

Parametres

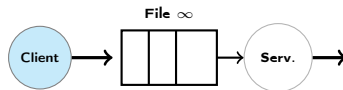
- 1 client, $\lambda \in \{4, 6, 8, 12\}$ messages/s
- 1 file (capacite infinie)
- 1 serveur, temps de service moyen $\mu = 8$ messages/s
- Mesurer toutes les metriques de base
- Tracer les resultats

Taux d'utilisation : $\rho = \lambda/\mu$

Nb moyen dans systeme : $L = \frac{\rho}{1 - \rho}$

Temps moyen dans systeme : $W = \frac{1}{\mu - \lambda}$

Condition de stabilite : $\rho < 1$, soit $\lambda < \mu = 8$.



λ	ρ	L	W (s)
4	0.5	1.0	0.250
6	0.75	3.0	0.500
8	1.0	∞	∞
12	1.5	∞	∞

Parametres communs

- 1 client, $\lambda \in \{4, 6, 8, 12\}$ messages/s
- 1 file, 1 serveur, $\mu = 8$ messages/s
- **Capacite maximale du systeme : K**
 - M/M/1/4 : au plus 4 messages dans le systeme
 - M/M/1/8 : au plus 8 messages dans le systeme
- Mesurer les metriques de base
- Inclure le **nombre de messages rejetes**
- Traces

Quand le systeme est plein (K messages), tout nouveau message est **rejete** (perdu).

Cela borne le nombre de messages dans le systeme :

- **Avantage** : le systeme est toujours stable (ρ peut etre > 1)
- **Inconvenient** : perte de messages

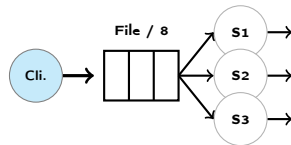
Quand λ augmente :

- Le taux de perte **augmente**
- Le temps d'attente **diminue** (moins de monde)
- M/M/1/8 perd moins que M/M/1/4 (file plus grande)

Simulation M/M/3/8 – file multi-serveurs

Parametres

- 1 client, $\lambda \in \{4, 6, 8, 12\}$ messages/s
- 1 file (partagee entre tous les serveurs)
- **3 serveurs**, $\mu = 8$ messages/s chacun
- Capacite max du systeme : 8
- Mesurer toutes les metriques + messages rejetes
- Traces



A λ egal, M/M/3/8 a :

- Moins de messages rejetes (capacite de traitement 3x)
- Temps d'attente plus court
- Taux d'utilisation par serveur = $\rho/3$

Modele	Serveurs	Capacite
M/M/1	1	∞
M/M/1/4	1	4
M/M/1/8	1	8
M/M/3/8	3	8

Evaluation des performances des systemes informatiques

Cours 09 – Simulateur

Pedro Braconnot Velloso & Abdelhak Hidouri